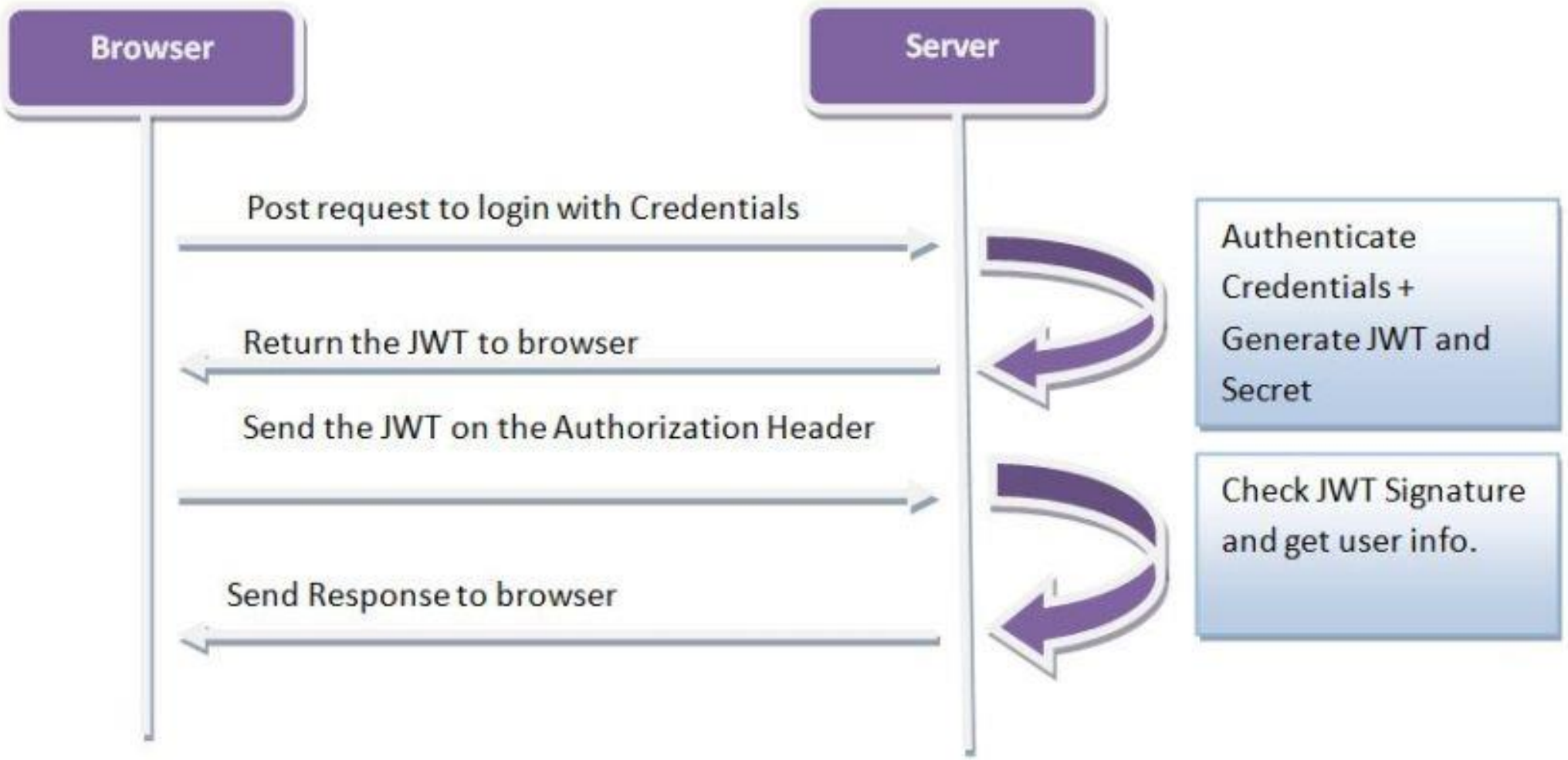
Authentication And Authorization In ASP.NET 5 WebApi With JWT And Swagger

Christen zarif

Authatication Based on Token



Introduction

- JSON Web Token (JWT)
  - Which is one of the most commonly used token based authentication.
  - It has become quite popular since it allows the distributed systems to securely communicate with each other without having the need to share the user id and password.

What is JSON Web Token?

- JWT stands for JSON Web Token. It is an Open standard (RFC 7519) which is used to securely transfer information between two parties.

- What is Open Standard? Open Standard means any one can use it without any restrictions.

- JWT is useful for Authorization and Information exchange.

Advantages of JWT

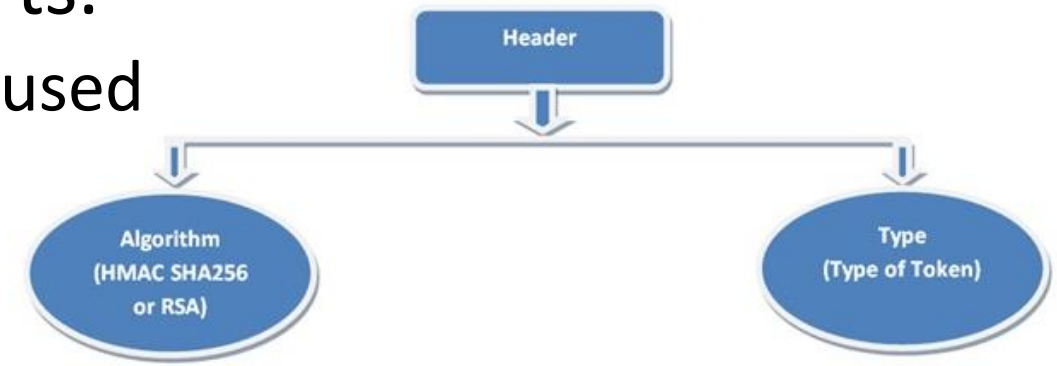
- **It is compact** – Can be sent via URL**/** Post request**/**HTTP Header which makes it fast for transmission and usable.
- **Self Content** – It contains the details of user (not session id in cookies like traditional request), so no need to query database to get user details.
- **Digitally signed** – Information is verified and trusted.
- **Transfer information between two bodies**. Bodies means user, sever etc./user, sever etc. is meant by Bodies.
- Can be used in most of the programming languages as it can be mapped directly to the object.
- JWTs can be signed using a secret (with the HMAC algorithm) or a public/private key pair using RSA or ECDSA
  - "HMAC" : **hash-based message authentication code**

JSON Web Token Structure

- JSON Web Token is combination of three parts with dots. These three parts are
  - Header
  - Payload
  - Signature.

Token - Head

- Header is a combination of two parts.
  - **Algorithm** – Signing algorithm being used such as HMAC SHA256 or RSA.
  - **Type** – Type of the token.




```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

This JSON is Base64Url encoded to form first part.

Token - Payload

- JSON object containing the information about the token i.e. the claims.
- Some of the claims are predefined based on the JWT standard. Custom claims can be added which are not part of the JWT standard.
- Even though anyone can see the claims, payload cannot be tampered since the JWT token becomes invalid if any of the claims are modified.

Predefine Claims

- Set of predefined claims which are not mandatory but recommended.
- Some of the registered claims are
  - “iss” (issuer) claim
  - “sub”(subject) claim
  - “aud” (Audience) claim
  - “exp”(Expiration Time) claim
  - “jit” (JWT ID) claim

Generate and verify a JWT with a symmetric key

- When a service generates a JWT, it also creates a signature. Traditionally, this signature is an HMAC,
- The HMAC takes the header, the payload and a secret key as input, and returns a unique signature over all three inputs.

- When a service receives an inbound JWT, it needs to verify the integrity before using the embedded data. To do so, the service uses the same secret key to calculate the HMAC of the JWT.

Authantication usin JWT

1-Install this Package

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.AspNetCore.Identity.EntityFrameworkCore
- Microsoft.AspNetCore.Authentication.JwtBearer

2-AppSetting.json

- We have added a database connection string and also added valid audience, valid issuer and secret key for JWT authentication in above settings file.

In appSetting.json

```
"ConnectionStrings": {
    "Cs": "Data Source=.;
           Initial Catalog=del;
           Integrated Security=True"
  },
"JWT": {
    "ValidAudience": "http://localhost:4200",
    "ValidIssuer": "http://localhost:62345",
    "Secret": "StrONGKAutHENTICATIONKEy"
  }
```

3- ApplicationUser and ApplicationDbContext

- Application User

```
public class ApplicationUser:IdentityUser
{

}
```

- ApplicationDbContext

```
public class ITIEntity:IdentityDbContext
{
    public ITIEntity()
    { }
    public ITIEntity(DbContextOptions options ):base(options)
    { }
}
```

4- Startup class – ConfigureService Method

1. Configure DbContext

```
services.AddDbContext<ITIEntity>(
    options => { options.UseSqlServer(Configuration.GetConnectionString("Cs")); });
```

2. Configure Identity

```
services.AddIdentity<ApplicationUser, IdentityRole>()
    .AddEntityFrameworkStores<ITIEntity>()
    .AddDefaultTokenProviders();
```

5- Add Authantication Service with JWTBearer

1. Configure Authantication //[Authorize]

```
services.AddAuthentication(options => {
        options.DefaultAuthenticateScheme =
                        JwtBearerDefaults.AuthenticationScheme;
        options.DefaultChallengeScheme=
                        JwtBearerDefaults.AuthenticationScheme;
        options.DefaultScheme =
                        JwtBearerDefaults.AuthenticationScheme;
    }).AddJwtBearer(options =>
```

JwtBearer Token options to validate

```
{
      options.SaveToken = true;
      options.RequireHttpsMetadata = false;
      options.TokenValidationParameters = new TokenValidationParameters()
      {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidAudience = Configuration["JWT:ValidAudience"],
        ValidIssuer = Configuration["JWT:ValidIssuer"],
        IssuerSigningKey = new
    SymmetricSecurityKey(Encoding.UTF8.GetBytes(Configuration["JWT:Secret"]))
      };
});
```

Migration

- Add-migration
- updatadatabase

Authantication Actions

- **Authenticate:** To authenticate basically means to use the given information and attempt to authenticate the user with that information. So this will attempt to create a user identity and make it available for the framework.
- For example, the cookie authentication scheme uses cookie data to restore the user identity. Or the JWT Bearer authentication scheme will use the token that is provided as part of the Authorization header in the request to create the user identity.

- **Challenge**: When an authentication scheme is challenged, the scheme should prompt the user to authenticate themselves. This could for example mean that the user gets redirected to a login form, or that there will be a redirect to an external authentication provider.

Configure Swaggerin ConfgiureService Method

```
services.AddSwaggerGen(swagger =>
            {
                //This is to generate the Default UI of Swagger Documentation
                swagger.SwaggerDoc("v1", new OpenApiInfo
                {
                    Version = "v1",
                    Title = "ASP.NET 5 Web API",
                    Description =  " ITI Projrcy"
                });
                // To Enable authorization using Swagger (JWT)
                swagger.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme()
                {
                    Name = "Authorization",
                    Type = SecuritySchemeType.ApiKey,
                    Scheme = "Bearer",
                    BearerFormat = "JWT",
                    In = ParameterLocation.Header,
                    Description = "Enter 'Bearer' [space] and then your valid token in the text input below.\r\n\r\nExample: \"Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6
IkpXVCJ9\"",
                });
```

Configure Swagger

```
    swagger.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
              new OpenApiSecurityScheme
                {
                    Reference = new OpenApiReference
                    {
                        Type = ReferenceType.SecurityScheme,
                        Id = "Bearer"
                    }
                },
                new string[] {}
        }
    });
});
```

In Configure Method

- Add this Middlewares

```
app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();
```

Create RegisterUserDto

```
public class RegisterUserDto
{
    [Required(ErrorMessage = "User Name is required")]
    public string Username { get; set; }

    [EmailAddress]
    [Required(ErrorMessage = "Email is required")]
    public string Email { get; set; }

    [Required(ErrorMessage = "Password is required")]
    public string Password { get; set; }
}
```

Create LoginUserDto

```
public class LoginDto
{
    [Required(ErrorMessage = "User Name is required")]
    public string Username { get; set; }

    [Required(ErrorMessage = "Password is required")]
    public string Password { get; set; }
}
```

Create AccountController

- Inject UserManager & Iconfigureation

```
private readonly UserManager<ApplicationUser> userManager;
private readonly IConfiguration configuration;

public AccountController(UserManager<ApplicationUser> userManager, IConfiguration configuration)
{
    this.userManager = userManager;
    this.configuration = configuration;
}
```

Register Action

- .

```
[HttpPost("Register")]
public async Task<IActionResult> Register(RegisterUserDto userDto)
{
    if (!ModelState.IsValid) {
        return BadRequest(ModelState);
    }
    var userExists = await userManager.FindByNameAsync(userDto.Username);
    if (userExists != null)
        return BadRequest("User already exists!" );

    ApplicationUser user = new ApplicationUser()
    {
        Email = userDto.Email,
        SecurityStamp = Guid.NewGuid().ToString(),
        UserName = userDto.Username
    };
    var result = await userManager.CreateAsync(user, userDto.Password);
    if (!result.Succeeded)
        return BadRequest(result.Errors.FirstOrDefault().Description);//.'

    return Ok("User created successfully!");

}
```

Login Action

```
public async Task<IActionResult> Login(LoginDto loginUser)
{
    if (!ModelState.IsValid) {
        return BadRequest(ModelState);
    }
    var user = await userManager.FindByNameAsync(loginUser.Username);
    if (user != null && await userManager.CheckPasswordAsync(user, loginUser.Password))
    {
        var userRoles = await userManager.GetRolesAsync(user);
        var authClaims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, user.UserName),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
        };
        foreach (var userRole in userRoles)
        {
            authClaims.Add(new Claim(ClaimTypes.Role, userRole));
        }
```

```
    var authSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(configuration["JWT:Secret"]));

    var token = new JwtSecurityToken(
        issuer: configuration["JWT:ValidIssuer"],
        audience: configuration["JWT:ValidAudience"],
        expires: DateTime.Now.AddHours(3),
        claims: authClaims,
        signingCredentials: new SigningCredentials(authSigningKey, SecurityAlgorithms.HmacSha256)
        );

    return Ok(new
    {
        token = new JwtSecurityTokenHandler().WriteToken(token),
        expiration = token.ValidTo
    });
}
return Unauthorized();
```

- Put [authorize] attribute above any controller
- Test using swagger or postman

- Test using Postman or Swagger

To Decode Token

- https://jwt.io/#debugger-io